Constants as the bridge

Minimal-shot autonomy, demonstrated on a snow plough

Snow-Underlay · SoTA Commission I · 2026-05-10

A snow plough's job is short: keep the road clear. The catch is that, while the plough is doing it, the road is invisible. Curbs are buried, lane markings are gone, the seam between asphalt and garden is no longer drawn. A self-driving stack trained on Cityscapes will report, with calibrated confidence, that the entire scene is sky.

The familiar response is *we just need more data*. Annotate snowy roads. Annotate dust storms. Annotate fog, lava, washouts. There are 27 million miles of road in the world, and the long tail of conditions any of them can be in is longer than the road itself. We are not going to label our way out of it.

There is a different move available. For almost every operating regime where autonomy fails for lack of data, there is an adjacent regime — temporally, seasonally, geographically — where we have plenty of data, and where the parts that matter are the same. A snow plough's road is the same road it was last July. The curb hasn't moved. The hydrant hasn't moved. The road's *appearance* has changed completely; its *position in space* has not.

If we can identify what stays constant between the data-rich regime and the data-poor one, we can extend our existing models into the new regime without learning a single thing about it. We use the constants as a bridge.

This essay is one concrete demonstration of that idea, applied to autonomous snow ploughs. The principle is general; the snow plough is the vehicle.

The example

Self-driving systems are trained on dry roads, deliberately. Cityscapes, KITTI, nuScenes, Waymo Open — the canonical training corpora — are dominated by clear-weather European or American highways under daylight. A snow plough operates in the regime that data deliberately excludes. Mistakes there damage infrastructure. The familiar move — collect more snowy data, annotate it, retrain — is uneconomic at the scale of a 27-million-mile road network.

But the road's *location* hasn't moved. For almost every road in the developed world there is a clear-season image of it: Street View, Mapillary, the operator's own prior captures. The plough's missing information is not what a road looks like under snow. It is *where this road sits in the camera frame right now*. That is a registration problem, not a learning problem. We solved registration twenty years ago.

So the system, in six steps:

1. Pull the live snowy frame from the plough's camera.

2. Pull a clear-season prior of the same coordinates from any open imagery substrate (we use Mapillary).
3. Match the two images using a generic, frozen feature matcher. The matcher anchors on what stays constant between the two: buildings, signs, poles, rooflines, distant horizons.
4. Estimate a homography from the matches, biased toward the ground plane.
5. Run a generic Cityscapes-trained road segmenter on the clear prior — never on the snow frame.
6. Warp the road mask through the homography onto the snowy frame.

The plough now knows where the road is, and where it isn't, even though it cannot see the road, and even though no model in the pipeline has ever been trained on a snowy frame.

Architecture

DISK (Tyszkiewicz et al., NeurIPS 2020) extracts local features from each image. LightGlue (Lindenberger et al., ICCV 2023) matches them. USAC-MAGSAC (Barath et al., CVPR 2020) fits a homography by RANSAC, restricted to lower-image matches to bias the fit toward the ground plane. When the initial fit is shaky, we re-fit on matches whose snow keypoints fall inside the warped road mask — an iterative segmentation-guided refinement that picks the road plane out of the wash of building façades. Mask2Former (Cheng et al., CVPR 2022), pretrained on Cityscapes (Cordts et al., CVPR 2016), produces the road mask on the clear prior. We reduce it (and its warp into snow image space) to its single largest connected component, because a plough cares about the *one* drivable surface in front of it. Open imagery comes from Mapillary's API v4, retrieved by image ID at run time.

Every learned component is frozen. Snow appears only at inference, as the runtime input.

What we showed

Fourteen hero pairs, manually rated GREAT or OKAY by a human reviewer after a five-stage curation funnel that started with 125 Mapillary candidates. Inlier counts in the GREAT bucket span 17–128, with the median at 38. Side-by-side with the same Cityscapes segmenter applied directly to the snow frame, the cross-season pipeline recovers the road; the direct application doesn't. The contrast is the demonstration.

The most interesting honest finding is one we did not expect: inlier count is not a reliable predictor of overlay quality. A pair with 238 inliers was rated NOT_GOOD because the inliers concentrated on building façades and the homography aligned the buildings rather than the road plane. A pair with 17 inliers was rated GREAT because the few inliers it had happened to land on the road. This is why the system needs a human in the loop on the input *and* the output, not just an automated metric. We shipped two small Streamlit raters for the curation work; the demo set is reproducible from `data/curated_pairs.json`.

What we didn't

We didn't train anything. We didn't fine-tune. We didn't collect a snow corpus. We didn't write a single line of snow-aware logic. The only handle we offered the model on the snow regime was the clear prior of the same place, plus the generic robustness of pretrained matchers and segmenters that have never seen snow.

We didn't claim the system replaces lidar or 3D depth estimation. The output is a 2D road *prior*, not a drivable-surface estimate. We didn't claim the homography is exact — it isn't, the world isn't planar — only that it transfers the road mask approximately and the approximation is enough.

We didn't claim novelty in any single component. The novelty, such as it is, is in the *composition*: the matcher, the segmenter, and the homography are off-the-shelf; the move is using them together to bridge a regime where one of them would otherwise fail.

What a real-time system would do

This is a proof of concept. A production snow plough would not match a single clear-season prior; it would fuse several. The frame edge of any one prior leaves a visible cutoff in the warped overlay; multiple priors at slightly different positions along the road wash that cutoff out. A bad single match can corrupt the result; a per-pixel consensus across K priors absorbs the bad match. The geometry is the same — match, homography, segment, warp — only repeated K times and combined.

We sketched this on the `multi-prior` branch of the repository (tag `v1.2-multi-prior-experiment`): adaptive 1–5 priors per query, three fusion strategies (union with edge erosion, weighted soft-average, hard majority vote), and a foreground crop. The empirical effect at our scale is real but small — a couple of demo pairs gained a level of subjective quality, a couple lost — and the explanatory cost in a 3-minute video is substantial. So we shipped the single-prior version as the demo. The fusion code is on the branch for anyone who wants to extend.

Generalising

The structure of the move is: a model trained on regime A; an inference target in regime B; a known correspondence between the two; transfer through the correspondence. Snow on a road is one instance. Others admit the same structure: low-light medical imaging without low-light training data, polar earth observation without polar training data, a manipulator on Mars without Mars training data. In each case there is a regime in which we have plenty of data, an adjacent regime in which we don't, and a constant between the two — temporal, geometric, anatomical — that lets us bridge.

We are not going to label our way out of every long-tail regime. But for many of them, we don't have to. We just have to find what stays the same and walk across.

Code, notebook, demo video, slides, and the curated demo set: see the project repository. Reproducible from a clean clone with a Mapillary API token via `make demo`*. Demo video music: "Slow Motion" by Bensound. Submitted to SoTA Commission I — Minimal-Shot Autonomy, May 2026.*